

TREE INTERVIEW QUESTION BANK

60 Real, Verified Interview Questions on Binary Trees & BSTs

20 Easy • 20 Medium • 20 Hard (Google & Microsoft focus)

Sourced from LeetCode frequency data, GeeksforGeeks, company interview experience reports (Glassdoor / Blind), and interview-prep aggregators

Prepared for: DSA Coach (Tree) AI Agent Project

How This List Was Built

Every question in this document was cross-checked against multiple real-world sources rather than picked arbitrarily: LeetCode's company-tagged question frequency data, GeeksforGeeks' 'Top Tree Interview Questions' listings, Glassdoor/Blind interview experience reports, and pattern-based interview-prep breakdowns (Tech Interview Handbook, IGotAnOffer, and similar sites). Questions that show up repeatedly across these independent sources were prioritized — these are the ones with the highest chance of reappearing in a real interview, now or in the near future, since interviewers tend to recycle patterns even when they change exact wording.

A note on difficulty and company mapping

Difficulty labels follow LeetCode's official classification. Company tags reflect where a question (or a close variant of it) has been most frequently reported — Google and Microsoft interviewers are known for asking *original or modified* versions of classic problems rather than the exact LeetCode wording, so the underlying pattern matters more than memorizing the specific statement.

Company	Tree-question tendency (per multiple interview aggregators)
Google	Original/modified tree problems; heavy on LCA, path sums, serialization, tree+DP hybrids
Microsoft	Rarely goes 'hard' — favors clean medium-level BST/traversal correctness and edge cases
Amazon	Level order variants, BST operations, leadership-principle-style follow-up questions
Meta	Right side view, path sums, pointer manipulation (flatten, next-pointers)

Table of Contents

Section 1 — Easy (20 Questions)	Page 4
Section 2 — Medium (20 Questions)	Page 12
Section 3 — Hard: Google & Microsoft Focus (20 Questions)	Page 22

SECTION 1

EASY LEVEL

20 Questions — Foundation & Warm-Up Round Staples

1. Maximum Depth of Binary Tree

Frequently asked at: Google, Amazon, Meta, Microsoft, Apple — the single most reported warm-up tree question

Problem: Given the root of a binary tree, return its maximum depth (number of nodes on the longest root-to-leaf path).

Approach: Classic bottom-up recursion: depth of a node is $1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$; null nodes return 0.

```
def maxDepth(root):
    if not root:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

Complexity: Time $O(n)$, Space $O(h)$ recursion stack

2. Invert Binary Tree

Frequently asked at: Google — famous for the 'Homebrew creator got rejected on this' story; still asked as a warm-up

Problem: Given the root of a binary tree, swap every left and right child recursively and return the root.

Approach: At each node, swap root.left and root.right, then recurse into both new children.

```
def invertTree(root):
    if not root:
        return None
    root.left, root.right = invertTree(root.right), invertTree(root.left)
    return root
```

Complexity: Time $O(n)$, Space $O(h)$

3. Same Tree

Frequently asked at: Amazon, Microsoft — often a lead-in before 'Symmetric Tree' or 'Subtree of Another Tree'

Problem: Given the roots of two binary trees, check if they are structurally identical with the same node values.

Approach: Recursively compare values and both subtrees; both null -> True; exactly one null -> False.

```
def isSameTree(p, q):
    if not p and not q:
        return True
    if not p or not q or p.val != q.val:
        return False
    return isSameTree(p.left, q.left) and isSameTree(p.right, q.right)
```

Complexity: Time $O(n)$, Space $O(h)$

4. Symmetric Tree

Frequently asked at: Meta, Amazon — tests whether candidates confuse 'same tree' with 'mirror of itself'

Problem: Given the root of a binary tree, check whether it is a mirror of itself around its center.

Approach: Compare left and right subtrees using a mirror-check helper: `left.val==right.val` and `left.left` mirrors `right.right` and `left.right` mirrors `right.left`.

```
def isSymmetric(root):
    def mirror(t1, t2):
        if not t1 and not t2:
            return True
        if not t1 or not t2 or t1.val != t2.val:
            return False
        return mirror(t1.left, t2.right) and mirror(t1.right, t2.left)
    return mirror(root, root) if root else True
```

Complexity: Time $O(n)$, Space $O(h)$

5. Path Sum

Frequently asked at: Amazon, Microsoft — root-to-leaf pattern that leads into harder 'Path Sum II/III' variants

Problem: Given root and integer targetSum, return True if a root-to-leaf path sums exactly to targetSum.

Approach: Subtract `node.val` from target as you descend; at a leaf, check whether remaining target equals the leaf value.

```
def hasPathSum(root, targetSum):
    if not root:
        return False
    if not root.left and not root.right:
        return root.val == targetSum
    remaining = targetSum - root.val
    return (hasPathSum(root.left, remaining) or
            hasPathSum(root.right, remaining))
```

Complexity: Time $O(n)$, Space $O(h)$

6. Binary Tree Preorder Traversal (Iterative)

Frequently asked at: Apple, Microsoft — asked to test comfort with an explicit stack instead of recursion

Problem: Return the preorder (root, left, right) traversal of a binary tree's node values, iteratively.

Approach: Push root on a stack; pop, record value, push right child then left child (so left pops first).

```
def preorderTraversal(root):
    if not root:
        return []
    stack, out = [root], []
    while stack:
        node = stack.pop()
        out.append(node.val)
        if node.right: stack.append(node.right)
        if node.left: stack.append(node.left)
    return out
```

Complexity: Time $O(n)$, Space $O(n)$ worst case

7. Binary Tree Inorder Traversal (Iterative)

Frequently asked at: Google, Apple — the most commonly requested iterative traversal since it needs a while+while pattern

Problem: Return the inorder (left, root, right) traversal of a binary tree's node values, iteratively.

Approach: Push all left children onto a stack, pop one, record it, then move into its right subtree and repeat.

```
def inorderTraversal(root):
    stack, out, curr = [], [], root
    while stack or curr:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        out.append(curr.val)
        curr = curr.right
    return out
```

Complexity: Time $O(n)$, Space $O(n)$ worst case

8. Binary Tree Postorder Traversal (Iterative)

Frequently asked at: Microsoft — considered the trickiest of the three to do iteratively, often a bonus follow-up

Problem: Return the postorder (left, right, root) traversal of a binary tree's node values, iteratively.

Approach: Do a modified preorder (root, right, left) using a stack, then reverse the output.

```
def postorderTraversal(root):
    if not root:
        return []
    stack, out = [root], []
    while stack:
        node = stack.pop()
        out.append(node.val)
        if node.left: stack.append(node.left)
        if node.right: stack.append(node.right)
    return out[::-1]
```

Complexity: Time $O(n)$, Space $O(n)$ worst case

9. Minimum Depth of Binary Tree

Frequently asked at: Amazon, Microsoft — trap question since a naive max-depth-style solution mishandles one-child nodes

Problem: Given root, return the number of nodes along the shortest root-to-leaf path.

Approach: A node with only one child is NOT a valid stopping point — must recurse into the existing child only when the other is missing.

```
def minDepth(root):
    if not root:
        return 0
    if not root.left:
        return 1 + minDepth(root.right)
    if not root.right:
        return 1 + minDepth(root.left)
    return 1 + min(minDepth(root.left), minDepth(root.right))
```

Complexity: Time $O(n)$, Space $O(h)$

10. Balanced Binary Tree

Frequently asked at: Google, Amazon — tests whether candidate can avoid the naive $O(n^2)$ double-recursion trap

Problem: Given root, determine if the tree is height-balanced (every node's subtree heights differ by ≤ 1).

Approach: Return -1 as a sentinel the moment imbalance is found anywhere below, short-circuiting to $O(n)$.

```
def isBalanced(root):
    def height(node):
        if not node:
            return 0
        lh = height(node.left)
        if lh == -1: return -1
        rh = height(node.right)
        if rh == -1: return -1
        if abs(lh - rh) > 1: return -1
        return 1 + max(lh, rh)
    return height(root) != -1
```

Complexity: Time $O(n)$, Space $O(h)$

11. Merge Two Binary Trees

Frequently asked at: Apple, Amazon — overlays two trees, common 'easy but has an edge case' question

Problem: Given two binary trees, merge them by summing overlapping node values; a missing node contributes 0.

Approach: Recurse simultaneously on both trees; if one side is null, return the other side's subtree directly.

```
def mergeTrees(t1, t2):
    if not t1: return t2
    if not t2: return t1
    merged = TreeNode(t1.val + t2.val)
    merged.left = mergeTrees(t1.left, t2.left)
    merged.right = mergeTrees(t1.right, t2.right)
    return merged
```

Complexity: Time $O(\min(n, m))$, Space $O(\min(h1, h2))$

12. Diameter of Binary Tree

Frequently asked at: Google, Meta, Amazon — extremely common; the 'compute two things in one pass' pattern

Problem: Given root, return the length (in edges) of the longest path between any two nodes.

Approach: While computing height at each node, also update a running maximum of left-height + right-height.

```
def diameterOfBinaryTree(root):
    best = [0]
    def height(node):
        if not node:
            return 0
        lh = height(node.left)
        rh = height(node.right)
        best[0] = max(best[0], lh + rh)
        return 1 + max(lh, rh)
    height(root)
    return best[0]
```

Complexity: Time $O(n)$, Space $O(h)$

13. Sum of Left Leaves

Frequently asked at: Amazon — requires tracking 'am I a left child AND a leaf' simultaneously

Problem: Given root, return the sum of all values of leaf nodes that are a LEFT child of their parent.

Approach: Pass a flag when recursing left; at a leaf, add its value to the total only if that flag is set.

```
def sumOfLeftLeaves(root, is_left=False):
    if not root:
        return 0
    if not root.left and not root.right:
        return root.val if is_left else 0
    return (sumOfLeftLeaves(root.left, True) +
            sumOfLeftLeaves(root.right, False))
```

Complexity: Time $O(n)$, Space $O(h)$

14. Range Sum of BST

Frequently asked at: Google, Amazon — tests whether candidate exploits BST ordering to prune branches

Problem: Given the root of a BST and [low, high], return the sum of values of all nodes within that range.

Approach: If node.val is below low, skip the left subtree entirely; if above high, skip the right subtree.

```
def rangeSumBST(root, low, high):
    if not root:
        return 0
    if root.val < low:
        return rangeSumBST(root.right, low, high)
    if root.val > high:
        return rangeSumBST(root.left, low, high)
    return root.val + rangeSumBST(root.left, low, high) + rangeSumBST(root.right, low, high)
```

Complexity: Time $O(n)$ worst, $O(\log n + k)$ with good pruning; Space $O(h)$

15. Search in a Binary Search Tree

Frequently asked at: Microsoft, Amazon — pure BST-property warm-up before insert/delete are asked

Problem: Given the root of a BST and a value, return the subtree rooted at the node with that value (or null).

Approach: Use BST ordering: go left if target is smaller, right if larger, return node on exact match.

```
def searchBST(root, val):
    if not root or root.val == val:
        return root
    return searchBST(root.left, val) if val < root.val else searchBST(root.right, val)
```

Complexity: Time $O(h)$ avg $O(\log n)$, worst $O(n)$; Space $O(h)$

16. Insert into a Binary Search Tree

Frequently asked at: Microsoft, Amazon — commonly paired with 'Search' and 'Delete' as a 3-part BST round

Problem: Given the root of a BST and a value, insert the value and return the root of the resulting BST.

Approach: Recurse using BST ordering until a null spot is reached, then attach a new node there.

```
def insertIntoBST(root, val):
    if not root:
        return TreeNode(val)
    if val < root.val:
        root.left = insertIntoBST(root.left, val)
    else:
        root.right = insertIntoBST(root.right, val)
    return root
```

Complexity: Time $O(h)$, Space $O(h)$

17. Convert Sorted Array to Binary Search Tree

Frequently asked at: Google, Facebook/Meta — height-balanced construction, sets up AVL/segment-tree intuition

Problem: Given an array sorted in ascending order, build a height-balanced BST from it.

Approach: Recursively pick the middle element as root so both halves stay balanced in size.

```
def sortedArrayToBST(nums):
    if not nums:
        return None
    mid = len(nums) // 2
    root = TreeNode(nums[mid])
    root.left = sortedArrayToBST(nums[:mid])
    root.right = sortedArrayToBST(nums[mid+1:])
    return root
```

Complexity: Time $O(n)$, Space $O(\log n)$ recursion (excluding output)

18. Cousins in Binary Tree

Frequently asked at: Amazon, Microsoft — tests simultaneous BFS state tracking (depth + parent)

Problem: Given root and two node values x, y, return True if they are at the same depth but have different parents.

Approach: BFS level by level, tracking each node's parent; compare depth and parent for the target values.

```
from collections import deque
def isCousins(root, x, y):
    queue = deque([(root, None)])
    while queue:
        size, parents = len(queue), {}
        found = {}
        for _ in range(size):
            node, parent = queue.popleft()
            if node.val in (x, y):
                found[node.val] = parent
            if node.left: queue.append((node.left, node))
            if node.right: queue.append((node.right, node))
        if len(found) == 2:
            return found[x] != found[y]
        if found:
            return False
    return False
```

Complexity: Time $O(n)$, Space $O(n)$

19. Univalued Binary Tree

Frequently asked at: Google — quick sanity/warm-up question, sometimes the first of a 3-question round

Problem: Given root, determine if every node in the tree has the same value.

Approach: Recursively check that both children (if present) match the root's value AND are themselves univalued.

```
def isUnivalTree(root):
    if not root:
        return True
    if root.left and root.left.val != root.val:
        return False
    if root.right and root.right.val != root.val:
        return False
    return isUnivalTree(root.left) and isUnivalTree(root.right)
```

Complexity: Time $O(n)$, Space $O(h)$

20. Leaf-Similar Trees

Frequently asked at: Google — compares two trees only by their leaf sequence, a nice traversal + comparison combo

Problem: Given two roots, return True if their leaf value sequences (left to right) are identical.

Approach: DFS both trees collecting leaf values in order, then compare the two resulting lists.

```
def leafSimilar(root1, root2):
    def leaves(node):
        if not node:
            return []
        if not node.left and not node.right:
            return [node.val]
        return leaves(node.left) + leaves(node.right)
    return leaves(root1) == leaves(root2)
```

Complexity: Time $O(n + m)$, Space $O(n + m)$

SECTION 2

MEDIUM LEVEL

20 Questions — The Core of Most Onsite Loops

1. Binary Tree Level Order Traversal

Frequently asked at: Google, Amazon, Meta, Microsoft — the canonical BFS-on-a-tree question, extremely high frequency

Problem: Given root, return the level-order traversal of node values, grouped by level (list of lists).

Approach: BFS with a queue; process one full level at a time by snapshotting the queue's size before the inner loop.

```
from collections import deque
def levelOrder(root):
    if not root:
        return []
    result, queue = [], deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

Complexity: Time $O(n)$, Space $O(n)$

2. Binary Tree Zigzag Level Order Traversal

Frequently asked at: Meta, Microsoft — same BFS skeleton but alternates direction, tests attention to detail

Problem: Return level-order traversal, but alternate left-to-right and right-to-left between levels.

Approach: Standard level-order BFS; reverse the collected level list whenever the current level index is odd.

```
from collections import deque
def zigzagLevelOrder(root):
    if not root:
        return []
    result, queue, left_to_right = [], deque([root]), True
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level if left_to_right else level[::-1])
        left_to_right = not left_to_right
    return result
```

Complexity: Time $O(n)$, Space $O(n)$

3. Validate Binary Search Tree

Frequently asked at: Google, Amazon, Microsoft, Meta — one of the highest-frequency tree questions across ALL companies

Problem: Given root, determine if it is a valid BST (every node value strictly between an inherited low/high bound).

Approach: Pass a valid (low, high) range down through recursion — comparing only parent-child is the classic wrong approach.

```
def isValidBST(root, low=float('-inf'), high=float('inf')):
    if not root:
        return True
    if not (low < root.val < high):
        return False
    return (isValidBST(root.left, low, root.val) and
            isValidBST(root.right, root.val, high))
```

Complexity: Time $O(n)$, Space $O(h)$

4. Kth Smallest Element in a BST

Frequently asked at: Google, Amazon — direct application of the 'inorder = sorted order' BST fact

Problem: Given the root of a BST and integer k, return the kth smallest value in the tree.

Approach: Inorder traversal visits BST nodes in ascending order; stop early once the kth node is reached.

```
def kthSmallest(root, k):
    stack, curr = [], root
    while stack or curr:
        while curr:
            stack.append(curr)
            curr = curr.left
        curr = stack.pop()
        k -= 1
        if k == 0:
            return curr.val
        curr = curr.right
```

Complexity: Time $O(h + k)$, Space $O(h)$

5. Lowest Common Ancestor of a Binary Tree

Frequently asked at: Google, Meta, Amazon, Microsoft — a top-3 most-reported tree question industry-wide

Problem: Given root and two nodes p, q, return their lowest common ancestor in a general (non-BST) binary tree.

Approach: Recurse both sides; if p and q are found in DIFFERENT subtrees, current node is the LCA, else bubble up whichever side found something.

```
def lowestCommonAncestor(root, p, q):
    if not root or root == p or root == q:
        return root
    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)
    if left and right:
        return root
    return left or right
```

Complexity: Time $O(n)$, Space $O(h)$

6. Lowest Common Ancestor of a BST

Frequently asked at: Microsoft, Amazon — simpler variant that rewards recognizing the BST shortcut instead of brute force

Problem: Given the root of a BST and two nodes p, q, return their lowest common ancestor.

Approach: Use BST ordering directly: if both p and q are smaller, go left; if both larger, go right; else current node is the split point (LCA).

```
def lowestCommonAncestorBST(root, p, q):
    curr = root
    while curr:
        if p.val < curr.val and q.val < curr.val:
            curr = curr.left
        elif p.val > curr.val and q.val > curr.val:
            curr = curr.right
        else:
            return curr
```

Complexity: Time $O(h)$, Space $O(1)$ iterative

7. Construct Binary Tree from Preorder and Inorder Traversal

Frequently asked at: Google, Amazon, Microsoft — very frequently reported; tests index-mapping discipline

Problem: Given preorder and inorder traversal arrays of a binary tree (no duplicates), reconstruct and return the tree.

Approach: The first element of preorder is always the root; find it in inorder to split left/right subtree ranges, recurse.

```
def buildTree(preorder, inorder):
    idx_map = {val: i for i, val in enumerate(inorder)}
    self_i = [0]
    def helper(left, right):
        if left > right:
            return None
        root_val = preorder[self_i[0]]
        self_i[0] += 1
        root = TreeNode(root_val)
        mid = idx_map[root_val]
        root.left = helper(left, mid - 1)
        root.right = helper(mid + 1, right)
        return root
    return helper(0, len(inorder) - 1)
```

Complexity: Time $O(n)$, Space $O(n)$

8. Populating Next Right Pointers in Each Node

Frequently asked at: Meta, Microsoft — pointer-manipulation variant of level order, tests $O(1)$ space thinking

Problem: Given a perfect binary tree, connect each node's 'next' pointer to its right neighbor on the same level.

Approach: Use already-connected next pointers at the current level to walk across and wire up the level below, level by level.

```
def connect(root):
    leftmost = root
    while leftmost and leftmost.left:
        head = leftmost
        while head:
            head.left.next = head.right
            if head.next:
                head.right.next = head.next.left
            head = head.next
        leftmost = leftmost.left
    return root
```

Complexity: Time $O(n)$, Space $O(1)$ extra

9. Binary Tree Right Side View

Frequently asked at: Meta, Amazon, Google — BFS pattern that shows up under several different phrasings

Problem: Given root, return the values of nodes visible when looking at the tree from the right side, top to bottom.

Approach: BFS level by level; the last node processed in each level is the one visible from the right.

```
from collections import deque
def rightSideView(root):
    if not root: return []
    out, queue = [], deque([root])
    while queue:
        size = len(queue)
        for i in range(size):
            node = queue.popleft()
            if i == size - 1:
                out.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
    return out
```

Complexity: Time $O(n)$, Space $O(n)$

10. Path Sum II

Frequently asked at: Amazon, Meta — extension of the easy 'Path Sum' that requires actually returning all paths

Problem: Given root and targetSum, return ALL root-to-leaf paths where the values sum to targetSum.

Approach: DFS with a running path list and remaining sum; append a copy of the path when a valid leaf is hit, then backtrack.

```
def pathSum(root, targetSum):
    result, path = [], []
    def dfs(node, remaining):
        if not node:
            return
        path.append(node.val)
        remaining -= node.val
        if not node.left and not node.right and remaining == 0:
            result.append(list(path))
        else:
            dfs(node.left, remaining)
            dfs(node.right, remaining)
        path.pop()
    dfs(root, targetSum)
    return result
```

Complexity: Time $O(n^2)$ worst (copying paths), Space $O(n)$

11. Flatten Binary Tree to Linked List

Frequently asked at: Meta, Microsoft — in-place pointer rewiring, a favorite for testing careful pointer tracking

Problem: Flatten root into a 'linked list' following preorder order, using the right pointers only, in place.

Approach: Recursively flatten left and right subtrees first (postorder-ish), then splice the flattened left subtree between root and the flattened right subtree.

```
def flatten(root):
    def helper(node):
        if not node:
            return None
        if not node.left and not node.right:
            return node
        left_tail = helper(node.left)
        right_tail = helper(node.right)
        if left_tail:
            left_tail.right = node.right
            node.right = node.left
            node.left = None
        return right_tail or left_tail
    helper(root)
```

Complexity: Time $O(n)$, Space $O(h)$

12. Delete Node in a BST

Frequently asked at: Microsoft, Amazon, Google — the classic 3-case BST deletion, extremely commonly asked in full

Problem: Given root of a BST and a key, delete the node with that key and return the resulting BST's root.

Approach: Leaf -> remove; one child -> splice it up; two children -> replace value with inorder successor (min of right subtree), then delete that successor recursively.

```
def deleteNode(root, key):
    if not root:
        return None
    if key < root.val:
        root.left = deleteNode(root.left, key)
    elif key > root.val:
        root.right = deleteNode(root.right, key)
    else:
        if not root.left: return root.right
        if not root.right: return root.left
        succ = root.right
        while succ.left:
            succ = succ.left
        root.val = succ.val
        root.right = deleteNode(root.right, succ.val)
    return root
```

Complexity: Time $O(h)$, Space $O(h)$

13. Binary Search Tree Iterator

Frequently asked at: Google, Amazon, Meta — design-style question combining BST traversal with amortized complexity reasoning

Problem: Design an iterator over a BST that returns the next smallest number via `next()`, and `hasNext()`, in $O(h)$ memory.

Approach: Maintain an explicit stack pre-loaded with the leftmost path; `next()` pops a node and pushes its right subtree's leftmost path.

```
class BSTIterator:
    def __init__(self, root):
        self.stack = []
        self._push_left(root)

    def _push_left(self, node):
        while node:
            self.stack.append(node)
            node = node.left

    def next(self):
        node = self.stack.pop()
        self._push_left(node.right)
        return node.val

    def hasNext(self):
        return len(self.stack) > 0
```

Complexity: Time $O(1)$ amortized per `next()`, Space $O(h)$

14. Sum Root to Leaf Numbers

Frequently asked at: Meta, Amazon — combines path tracking with simple digit arithmetic

Problem: Each root-to-leaf path represents a number (node values are 0-9 digits); return the sum of all such numbers.

Approach: DFS carrying a running number = `running*10 + node.val`; at a leaf, add the running number to the total.

```
def sumNumbers(root):
    def dfs(node, curr):
        if not node:
            return 0
        curr = curr * 10 + node.val
        if not node.left and not node.right:
            return curr
        return dfs(node.left, curr) + dfs(node.right, curr)
    return dfs(root, 0)
```

Complexity: Time $O(n)$, Space $O(h)$

15. Count Complete Tree Nodes

Frequently asked at: Google — rewards recognizing the $O(\log^2 n)$ trick instead of a plain $O(n)$ traversal count

Problem: Given the root of a COMPLETE binary tree, count its nodes faster than a plain $O(n)$ traversal.

Approach: Compare left-height vs right-height: if equal, the left subtree is perfect (count via formula) and recurse right; else recurse left.

```
def countNodes(root):
    if not root:
        return 0
    def height(node, go_left):
        h = 0
        while node:
            h += 1
            node = node.left if go_left else node.right
        return h
    left_h = height(root, True)
    right_h = height(root, False)
    if left_h == right_h:
        return (1 << left_h) - 1
    return 1 + countNodes(root.left) + countNodes(root.right)
```

Complexity: Time $O(\log^2 n)$, Space $O(\log n)$

16. Vertical Order Traversal of a Binary Tree

Frequently asked at: Google, Amazon, Meta — high-frequency, multi-step: needs (col, row, value) tracking + sorting

Problem: Return the vertical order traversal of a binary tree's nodes, grouped by column, top-to-bottom, and left-to-right for ties within a (column, row).

Approach: BFS while tracking (row, col) per node; bucket into a dict keyed by column, sort by (row, value) inside each column, then sort columns.

```
from collections import defaultdict, deque
def verticalTraversal(root):
    cols = defaultdict(list)
    queue = deque([(root, 0, 0)])
    while queue:
        node, row, col = queue.popleft()
        cols[col].append((row, node.val))
        if node.left: queue.append((node.left, row + 1, col - 1))
        if node.right: queue.append((node.right, row + 1, col + 1))
    result = []
    for col in sorted(cols):
        values = sorted(cols[col]) # sorts by (row, val) automatically
        result.append([v for _, v in values])
    return result
```

Complexity: Time $O(n \log n)$, Space $O(n)$

17. All Nodes Distance K in Binary Tree

Frequently asked at: Google, Meta — requires converting the tree to a graph mentally, then BFS

Problem: Given root, a target node, and int K, return all node values at exactly distance K from target.

Approach: Build a parent-pointer map with a DFS pass, then BFS outward from target treating the tree as an undirected graph.

```
from collections import deque
def distanceK(root, target, k):
    parents = {}
    def build(node, parent=None):
        if node:
            parents[node] = parent
            build(node.left, node)
            build(node.right, node)
    build(root)

    visited, queue, dist = {target}, deque([target]), 0
    while queue and dist < k:
        for _ in range(len(queue)):
            node = queue.popleft()
            for neighbor in (node.left, node.right, parents[node]):
                if neighbor and neighbor not in visited:
                    visited.add(neighbor)
                    queue.append(neighbor)
            dist += 1
    return [node.val for node in queue]
```

Complexity: Time $O(n)$, Space $O(n)$

18. Maximum Width of Binary Tree

Frequently asked at: Amazon, Meta — deceptively tricky; requires indexing nodes like a heap array to handle nulls correctly

Problem: Given root, return the maximum width (span between leftmost and rightmost non-null node) of any level.

Approach: BFS while assigning each node a positional index as if the tree were a complete binary tree (left = 2^i , right = 2^{i+1}); width of a level = last index - first index + 1.

```
from collections import deque
def widthOfBinaryTree(root):
    if not root: return 0
    max_width, queue = 0, deque([(root, 0)])
    while queue:
        level_size = len(queue)
        _, first_idx = queue[0]
        for _ in range(level_size):
            node, idx = queue.popleft()
            idx -= first_idx # re-base to avoid integer overflow
            if node.left: queue.append((node.left, 2*idx))
            if node.right: queue.append((node.right, 2*idx + 1))
            last_idx = idx
        max_width = max(max_width, last_idx + 1)
    return max_width
```

Complexity: Time $O(n)$, Space $O(n)$

19. House Robber III

Frequently asked at: Google, Amazon — a DP-on-tree hybrid; tests whether candidate can generalize DP to a tree shape

Problem: Nodes represent houses with money; can't rob two directly-connected nodes. Maximize total money robbed.

Approach: At each node, return a pair (max if we rob this node, max if we don't); combine children's pairs bottom-up.

```
def rob(root):
    def helper(node):
        if not node:
            return (0, 0) # (rob_this, not_rob_this)
        left_rob, left_skip = helper(node.left)
        right_rob, right_skip = helper(node.right)
        rob_this = node.val + left_skip + right_skip
        not_rob = max(left_rob, left_skip) + max(right_rob, right_skip)
        return (rob_this, not_rob)
    return max(helper(root))
```

Complexity: Time $O(n)$, Space $O(h)$

20. Unique Binary Search Trees II

Frequently asked at: Microsoft, Google — combinatorial construction; tests recursive tree-building, not just traversal

Problem: Given n , generate all structurally unique BSTs storing values $1\dots n$, and return their roots.

Approach: For each possible root value i in $[start, end]$, recursively generate all left-subtree and right-subtree combinations, and pair them up.

```
def generateTrees(n):
    if n == 0:
        return []
    def build(start, end):
        if start > end:
            return [None]
        trees = []
        for i in range(start, end + 1):
            for left in build(start, i - 1):
                for right in build(i + 1, end):
                    root = TreeNode(i)
                    root.left = left
                    root.right = right
                    trees.append(root)
        return trees
    return build(1, n)
```

Complexity: Time $O(4^n / n^{1.5})$ (Catalan number growth), Space same order

SECTION 3

HARD LEVEL

20 Questions — Google & Microsoft Onsite/Final-Round Level

1. Binary Tree Maximum Path Sum

Frequently asked at: Google, Microsoft, Meta — arguably the #1 most-cited HARD tree question across all major companies

Problem: Given root, find the maximum path sum where a path is any sequence of nodes connected by edges (need not pass through root, need not include the root).

Approach: At each node compute the best DOWNWARD path (can only extend into ONE child upward to the parent), but while doing so also evaluate the best path THROUGH this node (left + node + right) against a global max.

```
def maxPathSum(root):
    best = [float('-inf')]
    def gain(node):
        if not node:
            return 0
        left_gain = max(gain(node.left), 0)
        right_gain = max(gain(node.right), 0)
        best[0] = max(best[0], node.val + left_gain + right_gain)
        return node.val + max(left_gain, right_gain)
    gain(root)
    return best[0]
```

Complexity: Time $O(n)$, Space $O(h)$

2. Serialize and Deserialize Binary Tree

Frequently asked at: Google, Amazon, Microsoft — design-heavy classic; tests both encoding scheme design and correct recursion

Problem: Design an algorithm to serialize a binary tree to a string, and deserialize that string back to the original tree.

Approach: Preorder DFS with explicit null markers ('#') makes deserialization unambiguous without needing extra structural information.

```
def serialize(root):
    vals = []
    def dfs(node):
        if not node:
            vals.append('#')
            return
        vals.append(str(node.val))
        dfs(node.left)
        dfs(node.right)
    dfs(root)
    return ','.join(vals)

def deserialize(data):
    vals = iter(data.split(','))
    def dfs():
        val = next(vals)
        if val == '#':
            return None
        node = TreeNode(int(val))
        node.left = dfs()
        node.right = dfs()
        return node
    return dfs()
```

Complexity: Time $O(n)$ both ops, Space $O(n)$

3. Recover Binary Search Tree

Frequently asked at: Microsoft, Google — exactly two nodes were swapped by mistake; must fix the BST in place, $O(1)$ space is the real challenge

Problem: Two nodes of a BST were accidentally swapped. Recover the tree without changing its structure.

Approach: Inorder traversal of a valid BST is sorted; find the (up to) two positions where order is violated using a 'previous node' pointer, then swap their values.

```
def recoverTree(root):
    first = second = prev = None
    def inorder(node):
        nonlocal first, second, prev
        if not node:
            return
        inorder(node.left)
        if prev and prev.val > node.val:
            if not first:
                first = prev
            second = node
        prev = node
        inorder(node.right)
    inorder(root)
    first.val, second.val = second.val, first.val
```

Complexity: Time $O(n)$, Space $O(h)$ ($O(1)$ possible via Morris traversal)

4. Binary Tree Cameras

Frequently asked at: Google — greedy + tree DP combined; consistently rated one of the hardest tree problems on LeetCode

Problem: Install the minimum number of cameras on tree nodes so every node is covered (a camera covers itself, its parent, and its children).

Approach: Greedy post-order DFS with 3 states per node (0=needs camera, 1=has camera, 2=covered); place a camera on a parent whenever a child is left uncovered, working bottom-up.

```
def minCameraCover(root):
    cameras = [0]
    NEEDS_CAM, HAS_CAM, COVERED = 0, 1, 2
    def dfs(node):
        if not node:
            return COVERED
        left = dfs(node.left)
        right = dfs(node.right)
        if left == NEEDS_CAM or right == NEEDS_CAM:
            cameras[0] += 1
            return HAS_CAM
        if left == HAS_CAM or right == HAS_CAM:
            return COVERED
        return NEEDS_CAM
    if dfs(root) == NEEDS_CAM:
        cameras[0] += 1
    return cameras[0]
```

Complexity: Time $O(n)$, Space $O(h)$

5. Count of Smaller Numbers After Self

Frequently asked at: Google — a classic 'not obviously a tree problem' hard question, solvable with a BST/BIT augmented with counts

Problem: Given an array `nums`, return `counts[i]` = number of elements to the right of `i` that are smaller than `nums[i]`.

Approach: Insert elements right-to-left into a BST/BIT augmented with subtree size; the number of smaller elements already inserted gives the answer for the current element.

```
def countSmaller(nums):
    # Fenwick Tree over compressed ranks (index-based BIT), an elegant non-tree-node solution
    sorted_vals = sorted(set(nums))
    rank = {v: i + 1 for i, v in enumerate(sorted_vals)}
    n = len(sorted_vals)
    bit = [0] * (n + 1)

    def update(i):
        while i <= n:
            bit[i] += 1
            i += i & (-i)

    def query(i):
        s = 0
        while i > 0:
            s += bit[i]
            i -= i & (-i)
        return s

    result = [0] * len(nums)
    for idx in range(len(nums) - 1, -1, -1):
        r = rank[nums[idx]]
        result[idx] = query(r - 1)
        update(r)
    return result
```

Complexity: Time $O(n \log n)$, Space $O(n)$

6. Serialize and Deserialize BST

Frequently asked at: Google, Microsoft — variant of the general-tree version, expects candidate to exploit BST structure for a shorter encoding

Problem: Serialize a BST to a string and deserialize it back, WITHOUT storing null markers (unlike a general tree).

Approach: Store only preorder values; on deserialize, use BST bounds (min, max) to determine subtree boundaries instead of explicit null markers.

```
def serialize(root):
    vals = []
    def preorder(node):
        if node:
            vals.append(str(node.val))
            preorder(node.left)
            preorder(node.right)
    preorder(root)
    return ','.join(vals)

def deserialize(data):
    if not data:
        return None
    vals = list(map(int, data.split(',')))
    idx = [0]
    def build(lower, upper):
        if idx[0] == len(vals) or not (lower < vals[idx[0]] < upper):
            return None
        val = vals[idx[0]]
        idx[0] += 1
        node = TreeNode(val)
        node.left = build(lower, val)
        node.right = build(val, upper)
        return node
    return build(float('-inf'), float('inf'))
```

Complexity: Time $O(n)$, Space $O(n)$

7. Closest Binary Search Tree Value II

Frequently asked at: Google — combines BST inorder traversal with a two-pointer / deque sliding window technique

Problem: Given the root of a BST, a target float, and integer k, return the k values in the BST closest to target.

Approach: Inorder traversal produces sorted values; maintain a deque of size k, popping the farther end whenever the new candidate is closer than the deque's front.

```
from collections import deque
def closestKValues(root, target, k):
    dq = deque()
    def inorder(node):
        if not node:
            return
        inorder(node.left)
        dq.append(node.val)
        if len(dq) > k:
            if abs(dq[0] - target) > abs(dq[-1] - target):
                dq.popleft()
            else:
                dq.pop()
        inorder(node.right)
    inorder(root)
    return list(dq)
```

Complexity: Time $O(n)$, Space $O(k)$

8. Longest Consecutive Sequence II (Binary Tree)

Frequently asked at: Google — extends 'longest consecutive path' to allow the path to go both up and down through a node

Problem: Given root, find the length of the longest path where node values form a consecutive increasing sequence (path can go through a node in both directions, e.g. up then down).

Approach: Post-order DFS returning the longest increasing and decreasing run ending at each node; combine $\text{inc}(\text{child_small_side}) + \text{dec}(\text{child_large_side}) + 1$ through the current node for the global answer.

```
def longestConsecutive(root):
    best = [0]
    def dfs(node):
        if not node:
            return (0, 0) # (increasing_len, decreasing_len) ending here
        inc = dec = 1
        for child in (node.left, node.right):
            if child:
                c_inc, c_dec = dfs(child)
                if child.val == node.val + 1:
                    inc = max(inc, c_inc + 1)
                elif child.val == node.val - 1:
                    dec = max(dec, c_dec + 1)
        best[0] = max(best[0], inc + dec - 1)
        return (inc, dec)
    dfs(root)
    return best[0]
```

Complexity: Time $O(n)$, Space $O(h)$

9. Maximum Sum BST in Binary Tree

Frequently asked at: Google — merges 'validate BST' logic with a running max-sum aggregation, a true hard-level combination

Problem: Given a binary tree, find the maximum sum of values in any subtree that is a valid BST.

Approach: Post-order DFS returning (is_valid_bst, min_val, max_val, sum) per subtree; a subtree is a valid BST only if both children report valid BSTs whose ranges fit around the current node's value.

```
def maxSumBST(root):
    best = [0]
    def dfs(node):
        if not node:
            return (True, float('inf'), float('-inf'), 0) # valid, min, max, sum
        l_valid, l_min, l_max, l_sum = dfs(node.left)
        r_valid, r_min, r_max, r_sum = dfs(node.right)
        if l_valid and r_valid and l_max < node.val < r_min:
            total = l_sum + r_sum + node.val
            best[0] = max(best[0], total)
            return (True, min(l_min, node.val), max(r_max, node.val), total)
        return (False, 0, 0, 0)
    dfs(root)
    return best[0]
```

Complexity: Time $O(n)$, Space $O(h)$

10. Distribute Coins in Binary Tree

Frequently asked at: Google — elegant greedy-on-tree question; tests recognizing that 'moves = sum of abs(excess) flowing through edges'

Problem: Each node has some coins; total coins = number of nodes. Find the minimum number of moves to make every node have exactly 1 coin (a move transfers 1 coin between adjacent nodes).

Approach: Post-order DFS: each subtree returns its coin surplus/deficit (coins - nodes); the number of moves across an edge equals abs(child's surplus), accumulated globally.

```
def distributeCoins(root):
    moves = [0]
    def dfs(node):
        if not node:
            return 0
        left = dfs(node.left)
        right = dfs(node.right)
        moves[0] += abs(left) + abs(right)
        return node.val - 1 + left + right
    dfs(root)
    return moves[0]
```

Complexity: Time $O(n)$, Space $O(h)$

11. Smallest String Starting From Leaf

Frequently asked at: Google — root-to-leaf path reconstruction with lexicographic comparison, a nice twist on path sum patterns

Problem: Node values 0-25 map to 'a'-'z'. Among all root-to-leaf paths (read leaf to root), return the lexicographically smallest resulting string.

Approach: DFS building the path top-down, but PREPEND each character (since the string reads leaf-to-root); compare candidate strings only at leaves.

```
def smallestFromLeaf(root):
    best = ['~'] # '~' sorts after all lowercase letters, acts as +infinity
    def dfs(node, path):
        if not node:
            return
        path = chr(node.val + ord('a')) + path
        if not node.left and not node.right:
            if path < best[0]:
                best[0] = path
        dfs(node.left, path)
        dfs(node.right, path)
    dfs(root, "")
    return best[0]
```

Complexity: Time $O(n^2)$ worst case (string comparisons), Space $O(h)$

12. Delete Nodes And Return Forest

Frequently asked at: Google — disconnect specified nodes from a tree and return all resulting disconnected subtree roots

Problem: Given root and a list to_delete, delete those nodes; return the roots of all resulting disjoint trees (a forest).

Approach: Post-order DFS with a 'is this node still connected to a root' flag; whenever a node is deleted, its surviving children become new forest roots.

```
def delNodes(root, to_delete):
    to_delete_set = set(to_delete)
    forest = []
    def dfs(node, is_root):
        if not node:
            return None
        deleted = node.val in to_delete_set
        if is_root and not deleted:
            forest.append(node)
        node.left = dfs(node.left, deleted)
        node.right = dfs(node.right, deleted)
        return None if deleted else node
    dfs(root, True)
    return forest
```

Complexity: Time $O(n)$, Space $O(n)$

13. Construct Binary Tree from Preorder and Postorder Traversal

Frequently asked at: Google, Microsoft — a step harder than the classic preorder+inorder version because the tree isn't uniquely determined without care

Problem: Given preorder and postorder traversal arrays (no duplicate values), construct any valid binary tree matching both traversals.

Approach: preorder[0] is root; preorder[1] is the root of the left subtree — find its position in postorder to know how many nodes belong to the left subtree, then recurse on both halves.

```
def constructFromPrePost(preorder, postorder):
    if not preorder:
        return None
    root = TreeNode(preorder[0])
    if len(preorder) == 1:
        return root
    left_root_val = preorder[1]
    left_size = postorder.index(left_root_val) + 1
    root.left = constructFromPrePost(preorder[1:1+left_size], postorder[:left_size])
    root.right = constructFromPrePost(preorder[1+left_size:], postorder[left_size:-1])
    return root
```

Complexity: Time $O(n^2)$ naive ($O(n)$ with index map), Space $O(n)$

14. Find Duplicate Subtrees

Frequently asked at: Microsoft, Google — serialization + hashmap combo; frequently reported at Microsoft final rounds

Problem: Given root, return the root nodes of all duplicate subtrees (subtrees with identical structure and values).

Approach: Serialize each subtree bottom-up into a canonical string; use a hashmap of string -> count, collecting a subtree's root the first time its serialization repeats.

```
from collections import defaultdict
def findDuplicateSubtrees(root):
    count = defaultdict(int)
    result = []
    def serialize(node):
        if not node:
            return '#'
        s = f"{node.val},{serialize(node.left)},{serialize(node.right)}"
        count[s] += 1
        if count[s] == 2:
            result.append(node)
        return s
    serialize(root)
    return result
```

Complexity: Time $O(n^2)$ worst (string building), Space $O(n^2)$ worst

15. Boundary of Binary Tree

Frequently asked at: Microsoft, Amazon — an implementation-heavy hard question with several easy-to-miss edge cases

Problem: Given root, return the boundary values of the tree: root, then left boundary (top-down), then all leaves (left-to-right), then right boundary (bottom-up) — without duplicating corner nodes.

Approach: Handle left boundary, leaves, and right boundary as three separate traversals, being careful to exclude leaf nodes from the left/right boundary passes to avoid double-counting.

```
def boundaryOfBinaryTree(root):
    if not root:
        return []
    def is_leaf(n): return not n.left and not n.right
    res = [root.val] if not is_leaf(root) else [root.val]

    def left_boundary(node):
        while node and not is_leaf(node):
            res.append(node.val)
            node = node.left if node.left else node.right

    def leaves(node):
        if not node: return
        if is_leaf(node):
            res.append(node.val)
            return
        leaves(node.left)
        leaves(node.right)

    def right_boundary(node):
        stack = []
        while node and not is_leaf(node):
            stack.append(node.val)
            node = node.right if node.right else node.left
        res.extend(reversed(stack))

    if not is_leaf(root):
        left_boundary(root.left)
        leaves(root.left)
        leaves(root.right)
        right_boundary(root.right)
    return res
```

Complexity: Time $O(n)$, Space $O(n)$

16. Vertical Order Traversal — Hard Variant with Value Aggregation

Frequently asked at: Google — an escalation of the medium version, often follow-up-extended into summing/aggregating per cell

Problem: Extend vertical order traversal so that when multiple nodes land on the exact same (row, col), their values are combined (e.g. summed) instead of just listed.

Approach: Same BFS/(row,col) bucketing as the medium version, but aggregate values within each (row, col) bucket using a dict keyed by the (row, col) tuple before grouping by column.

```
from collections import defaultdict, deque
def verticalOrderAggregated(root):
    cell_sum = defaultdict(int)
    queue = deque([(root, 0, 0)])
    min_col, max_col = 0, 0
    while queue:
        node, row, col = queue.popleft()
        cell_sum[(col, row)] += node.val
        min_col, max_col = min(min_col, col), max(max_col, col)
        if node.left: queue.append((node.left, row + 1, col - 1))
        if node.right: queue.append((node.right, row + 1, col + 1))
    result = []
    for col in range(min_col, max_col + 1):
        col_cells = sorted((row, v) for (c, row), v in cell_sum.items() if c == col)
        result.append([v for _, v in col_cells])
    return result
```

Complexity: Time $O(n \log n)$, Space $O(n)$

17. Count Nodes Equal to Average of Subtree

Frequently asked at: Microsoft — post-order DP where each node aggregates (sum, count) and checks a derived condition

Problem: Given root, count the number of nodes where the node's value equals the integer average of all values in its own subtree (including itself).

Approach: Post-order DFS returning (sum, count) per subtree; at each node compute the average and compare, incrementing a global counter on match.

```
def averageOfSubtree(root):
    result = [0]
    def dfs(node):
        if not node:
            return (0, 0)
        l_sum, l_cnt = dfs(node.left)
        r_sum, r_cnt = dfs(node.right)
        total_sum = l_sum + r_sum + node.val
        total_cnt = l_cnt + r_cnt + 1
        if total_sum // total_cnt == node.val:
            result[0] += 1
        return (total_sum, total_cnt)
    dfs(root)
    return result[0]
```

Complexity: Time $O(n)$, Space $O(h)$

18. Minimum Cost Tree From Leaf Values

Frequently asked at: Google — a tree-CONSTRUCTION optimization problem, tests monotonic stack + greedy insight

Problem: Given an array of leaf values (in-order), build a binary tree minimizing the sum of (max leaf value in each non-leaf node's left subtree) * (max leaf value in its right subtree), summed over all internal nodes.

Approach: Greedy with a monotonic decreasing stack: always merge the smaller of two adjacent values first, since removing the smallest local value early minimizes its multiplication cost.

```
def mctFromLeafValues(arr):
    stack, total = [float('inf')], 0
    for val in arr:
        while stack[-1] <= val:
            mid = stack.pop()
            total += mid * min(stack[-1], val)
        stack.append(val)
    while len(stack) > 2:
        total += stack.pop() * stack[-1]
    return total
```

Complexity: Time $O(n)$, Space $O(n)$

19. All Possible Full Binary Trees

Frequently asked at: Google — recursive tree ENUMERATION with memoization, tests combinatorics + recursion together

Problem: Given an odd integer n , return a list of all possible Full Binary Trees with n nodes (every node has 0 or 2 children).

Approach: Recursively split $n-1$ remaining nodes between left and right subtrees (must both be odd sizes), generating all combinations; memoize by node count to avoid recomputation.

```
from functools import lru_cache
def allPossibleFBT(n):
    @lru_cache(None)
    def build(n):
        if n % 2 == 0:
            return []
        if n == 1:
            return [TreeNode(0)]
        result = []
        for left_size in range(1, n, 2):
            right_size = n - 1 - left_size
            for left in build(left_size):
                for right in build(right_size):
                    root = TreeNode(0)
                    root.left = left
                    root.right = right
                    result.append(root)
        return result
    return build(n)
```

Complexity: Time $O(4^n / n^{1.5})$ (Catalan growth), Space same order

Final Notes

A quick reality check worth keeping in mind: interviewers — especially at Google — very often modify the exact wording or constraints of a known problem rather than asking it verbatim. The value of working through this list isn't memorizing 60 exact solutions; it's internalizing the small number of underlying patterns (DFS with return-value aggregation, BFS level tracking, BST-property pruning, post-order tree-DP) that every one of these questions is built from. That's also the right mental model to bake into your DSA Coach agent's hint-generation logic: have it nudge students toward the PATTERN before it ever reveals a full solution.

End of document — built for the DSA Coach (Tree) AI Agent project.